

Express.js - Day 4

Route Parameters & Query Parameters

Day 4 Goal

Understand how Express reads values from a URL using route parameters and query parameters, and know when each is useful.

1. Route Parameters - req.params

A route parameter is a value that is part of the URL path. It is defined with a colon (:).

```
app.get('/users/:id', (req, res) => {  
  console.log(req.params)  
  res.send('User found')  
})
```

For /users/100, Express matches :id with 100.

```
Result: { id: '100' }
```

2. Accessing One Route Parameter

Use the parameter name from the route definition.

```
const id = req.params.id
```

If the route uses :userId, use req.params.userId.

3. Multiple Route Parameters

```
app.get('/users/:userId/posts/:postId', (req, res) => {  
  console.log(req.params)  
  res.send('Post found')  
})
```

For /users/100/posts/10:

```
{ userId: '100', postId: '10' }
```

4. Query Parameters - req.query

Query parameters are additional values placed after a ? in the URL. They are commonly used for filtering, sorting, searching, pagination, and optional settings.

```
/products/50?category=phone&sort=price
```

Here, 50 is a route parameter. category and sort are query parameters.

```
req.query -> { category: 'phone', sort: 'price' }
```

5. Multiple Query Parameters

Use & to separate query parameters.

```
/products?category=phone&sort=price&limit=10
```

```
{ category: 'phone', sort: 'price', limit: '10' }
```

6. Route Parameters vs Query Parameters

Feature	Route Parameters	Query Parameters
Property	req.params	req.query
Location	Part of path	After ?
Example	/users/25	/users?age=25
Purpose	Identify a resource	Filter / sort / optional data

7. Both Can Be Used Together

```
GET /products/50?category=phone&sort=price
```

```
req.params -> { id: '50' }
```

```
req.query -> { category: 'phone', sort: 'price' }
```

8. URL Values Are Strings by Default

Values coming from URL parameters are strings by default, even when they look like numbers.

```
For /users/25:  
req.params.id -> '25'  
typeof req.params.id -> 'string'
```

If a numeric value is required, convert it explicitly:

```
const id = Number(req.params.id)
```

9. Practical Example

For an e-commerce API, /products/50?category=phone&sort=price could mean: product ID = 50, category = phone, and sorting = price.

Day 4 Mental Model

```
req.params -> Which resource?
```

```
req.query -> How should I filter, sort, search, or customize the result?
```

Day 4 Assessment - Answers

1. /users/25 uses a route parameter because 25 is part of the path. /users?age=25 uses a query parameter because age=25 is additional information after ?.

2. For /products/100?category=laptop&sort=price:

```
req.params -> { id: '100' }
```

```
req.query -> { category: 'laptop', sort: 'price' }
```

3. In /users/:id, :id is the dynamic route parameter. For /users/25, its value is '25'.

4. No. URL parameters are strings by default. Use Number(req.params.id) when a numeric value is required.

Quick Revision

Route parameter -> part of the path -> req.params

Query parameter -> after ? -> req.query

Multiple query parameters -> separated by &

Both can exist in one request.

URL values are strings by default.

Convert numeric values explicitly when needed.